LP - III

Mini Project Report

On

Decentralized dApp E-Voting System

Submitted in the fulfillment of requirements of Savitribai Phule Pune University

For the degree of **Third Year of Computer Engineering**

AY 2026-27

By

Mayur Zope TECA171

Yash Chaudhary TECA170

Prem Borde TECA111

Under the guidance of

Prof. Akansha Damonkar



Department of Computer Engineering

PCET-NMVPM's

NUTAN MAHARASHTRA INSTITUTE OF ENGINEERING AND TECHNOLOGY

TALEGAON-DABHADE,

Vishnupuri, Talegaon Station-410507

SAVITRIBAI PHULE PUNE UNIVERSITY



NUTAN MAHARASHTRA INSTITUTE OF ENGINEERING & TECHNOLOGY, TALEGAON-DABHADE

CERTIFICATE

This is to Certify that -

Name	PRN
Mayur Zope	72322283F
Yash Chaudhary	72322276C
Prem Borde	72321957F

Has satisfactorily completed the requirements of Project under LP-III - Blockchain Mini Project

For the degree of **Computer Engineering**

On –

Decentralized dApp E-Voting System

Prof. Akansha Damonkar	Dr. Rahul Patil	Dr. Pramod Patil
Course Coordinator	Head of Department	Director

Acknowledgement

We have taken efforts in this project. However, it would not have been possible without the kind support and help of many individuals and organizations. We would like to extend our sincere thanks to all of them.

We are highly indebted to Prof. Sonal Kadam for her guidance and constant supervision, as well as for providing the necessary information regarding the project and for her support in completing it.

We would like to thank our HOD, Dr. Rahul Patil, for his constant support in all project-related activities.

We would like to express our gratitude towards our parents and the members of NMIET for their kind co-operation and encouragement, which helped us in the completion of this project.

Our thanks and appreciation also go to our colleagues who helped us in developing the project and to everyone who willingly helped us with their abilities.

Abstract

Blockchain technology has changed the way trust is established in digital systems by replacing central authorities with a distributed, tamper-resistant ledger. This project presents the development of a decentralized application (dApp) titled **Decentralized dApp E-Voting System** using the Solidity programming language and Ethereum-compatible blockchain technology.

The main objective of the application is to record votes transparently and to guarantee that a single account cannot vote more than once. A Solidity smart contract named `Voting.sol` stores the list of candidates, their vote counts, and a record of the addresses that have already voted. The contract is written, compiled, and deployed using Remix IDE on the Remix VM, and can be accessed through a web frontend built with HTML, CSS, and JavaScript, connected to the blockchain through the MetaMask wallet.

Because votes are stored on the blockchain, they cannot be altered after being recorded, and anyone can verify the results. This project demonstrates the practical implementation of smart contracts, decentralized storage, and wallet-based identity.

Keywords: Blockchain, Ethereum, Solidity, Smart Contract, dApp, E-Voting, Remix IDE, MetaMask

CONTENTS

Chapter	Title
I	Certificate
II	Acknowledgement
III	Abstract
1	Problem Statement
2	Software and Hardware Requirements
3	Proposed Methodology
4	Design & Theory
5	Code & Output
6	Conclusion
7	References

CHAPTER 1: PROBLEM STATEMENT

In many organizations, colleges, and institutions, elections are still conducted using paper ballots or centralized electronic systems. These approaches depend on a central authority that must be trusted to count and report votes honestly, and they are exposed to several weaknesses.

The major problems associated with traditional voting systems include:

- **Lack of Transparency:** Voters cannot independently verify that their vote was recorded and counted correctly.
- **Risk of Tampering:** Records held in a central database can be modified or deleted by an administrator or an attacker.
- **Duplicate Voting:** Without reliable identity checks, the same person may be able to vote more than once.
- **Single Point of Failure:** A central server can be attacked, can crash, or can be shut down, disrupting the election.
- **Slow Counting and Human Error:** Manual counting is time-consuming and error-prone.

With the growing need for trustworthy digital processes, there is a strong requirement for a system in which the rules of the election are enforced by code that nobody can silently change.

Therefore, this project proposes the development of a **Decentralized dApp E-Voting System** using Solidity and Ethereum-compatible blockchain technology. The system aims to record votes on a distributed ledger, prevent any address from voting more than once, and let anyone verify the vote counts in real time.

CHAPTER 2: SOFTWARE / HARDWARE REQUIREMENTS

2.1 Software Requirements

Component	Details
Smart Contract Language	Solidity (version ^0.8.20)
Development and Deployment	Remix IDE
Blockchain	Remix VM (local testing) or an Ethereum-compatible test network
Wallet	MetaMask browser extension
Frontend	HTML, CSS, and JavaScript
OS	Windows / Ubuntu
Browser	Google Chrome

For the reference web portal described in Section 4.6, the following are additionally used: React.js, Truffle, Ganache, and Web3.js.

2.2 Hardware Requirements

Component	Specification
Processor	Intel i3 or above
RAM	Minimum 4 GB
Storage	500 MB free space
Network	Internet connection (for Remix IDE and test networks)

CHAPTER 3: PROPOSED METHODOLOGY

3.1 Methodology Overview

The project follows a structured approach starting from requirement analysis and smart contract design to deployment and testing on a blockchain. Voting rules are written as a Solidity smart contract, which is then deployed using Remix IDE and tested with multiple accounts.

3.2 Steps Followed

1. **Open Remix IDE and create the contract file** A new file named `Voting.sol` is created in Remix IDE.
2. **Write the smart contract** The contract defines the owner, a `Candidate` structure, an array of candidates, and a mapping that records which addresses have voted.
3. **Compile the smart contract** The contract is compiled using the Solidity compiler in Remix to check for errors.
4. **Deploy the contract** The contract is deployed using Remix VM, providing candidate names such as `["Alice", "Bob"]` to the constructor.
5. **Cast votes** The `vote` function is called with `0` to vote for Alice or `1` to vote for Bob.
6. **Test with another account** Another test account is selected in Remix to cast a second vote, and a repeated vote from the same account is attempted to verify that it is rejected.
7. **View results** The `getCandidate` function is called to view each candidate's vote count.
8. **Frontend and wallet integration** A frontend built with HTML, CSS, and JavaScript connects to the deployed contract through MetaMask so that users can vote from a web page.

3.3 Workflow of the System

- The deployer publishes the contract with the list of candidate names.
- A voter connects a wallet account and selects a candidate.
- The voter submits the `vote` transaction.
- The contract checks that the address has not voted and that the candidate is valid.
- The vote is recorded on the blockchain and the address is marked as having voted.
- Anyone can read the candidate vote counts at any time.

3.4 Methodology Advantages

- **Transparency:** All votes and counts are publicly verifiable.
- **Immutability:** Recorded votes cannot be changed or deleted.

- **No Central Authority:** Rules are enforced by the smart contract, not by an administrator.
- **Duplicate Prevention:** Each address can vote only once.
- **Fast Development and Testing:** Remix IDE allows quick compilation, deployment, and testing without any local setup.

CHAPTER 4: DESIGN & THEORY

4.1 System Architecture

The system follows a decentralized architecture in which the application logic runs on the blockchain rather than on a private server.

Architecture Components:

- **Frontend:** HTML, CSS, and JavaScript web interface
- **Wallet:** MetaMask, used to sign and send transactions
- **Smart Contract:** `Voting.sol` written in Solidity
- **Blockchain:** Remix VM or an Ethereum-compatible test network
- **Development Tool:** Remix IDE for writing, compiling, and deploying

Working: The user interacts with the web page, which asks MetaMask to sign a transaction. The transaction is sent to the blockchain, where the smart contract executes the voting logic and updates its stored data. The frontend then reads the updated vote counts from the contract.

4.2 Theory

- **Blockchain:** A distributed ledger in which records are grouped into blocks that are cryptographically linked, making past records practically impossible to alter.
- **Ethereum:** A blockchain platform that can run programs called smart contracts.
- **Smart Contract:** Code stored on the blockchain that executes automatically when its functions are called and whose rules cannot be changed after deployment.
- **Solidity:** A high-level language used to write smart contracts for the Ethereum Virtual Machine.
- **Gas:** The fee paid for executing transactions that change blockchain state. Read-only calls such as `getCandidate` do not cost gas.
- **Wallet Address:** A unique identifier derived from a public key, used here to identify a voter.
- **dApp:** A decentralized application whose backend logic runs as smart contracts on a blockchain.

4.3 Smart Contract Design

Element	Type	Purpose
owner	address	Stores the address that deployed the contract

Element	Type	Purpose
Candidate	struct	Holds a candidate's name and votes
candidates	Candidate[]	List of all candidates
hasVoted	mapping(address => bool)	Records which addresses have already voted
constructor	function	Creates the candidates from the supplied names
vote	function	Records a vote after validation
getCandidateCount	view function	Returns the number of candidates
getCandidate	view function	Returns a candidate's name and votes

4.4 Validation Rules

- require(!hasVoted[msg.sender], "Already voted") rejects a second vote from the same address.
- require(candidateIndex < candidates.length, "Invalid candidate") rejects a vote for a candidate that does not exist.

4.5 Design Features

- **Decentralized Storage:** Vote data lives on the blockchain.
- **Transparent Results:** Vote counts can be read by anyone.
- **One Address, One Vote:** Enforced by the contract itself.
- **Wallet-Based Identity:** Voters are identified by their wallet address.

4.6 Reference Web Portal: Electronic Voting System

The web portal used for the output screens in Chapter 5 is taken from a reference implementation [6]. It is a decentralized electronic voting system that lets users cast votes through a web portal in an efficient and secure manner. It aims to change how voting works by using blockchain, which is immutable (a vote cannot be undone once cast), decentralized and distributed (no central authority controls the voting system), and therefore transparent.

Working:

1. The voter first logs in to the dApp using their Aadhaar number and OTP confirmation.

2. After authentication, the voter can begin the voting process.
3. If the voter is valid, the voting palette opens with the candidate names.
4. The voter casts a vote by clicking the Vote button next to the preferred candidate.
5. A voter can vote only once, after which the voter is automatically logged out.
6. The same process continues for any number of further voters, irrespective of their voting wards.
7. The vote count for each candidate is displayed on the portal, next to the candidate.

Advantages:

- No voter ID is required, as the user's validity (age 18+) is determined using Aadhaar.
- Reduced cost during the election process.
- Shorter wait times, as the system is decentralized.
- A vote can be cast from anywhere in the country.
- Highly scalable design.
- The portal can stay up for days, which can increase voter turnout.
- The frontend can be extended to show candidate information such as promises and proposals, aiding voters in decision making.

Technology Stack:

1. React.js
2. Truffle
3. Ganache
4. MetaMask
5. Web3.js

4.7 Limitations

This is a basic implementation intended to demonstrate core concepts. On its own, the contract does not verify that an address belongs to a real, eligible person, so one person with multiple wallets could vote more than once. Votes are also not secret, since transactions are public. The `owner` is stored but not yet used to restrict any action.

CHAPTER 5: CODE & OUTPUT

5.1 Smart Contract Code (Voting.sol)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract Voting {
    address public owner;

    struct Candidate {
        string name;
        uint256 votes;
    }

    Candidate[] public candidates;
    mapping(address => bool) public hasVoted;

    constructor(string[] memory candidateNames) {
        owner = msg.sender;

        for (uint256 i = 0; i < candidateNames.length; i++) {
            candidates.push(
                Candidate(candidateNames[i], 0)
            );
        }
    }

    function vote(uint256 candidateIndex) public {
        require(!hasVoted[msg.sender], "Already voted");
        require(
            candidateIndex < candidates.length,
            "Invalid candidate"
        );

        hasVoted[msg.sender] = true;
        candidates[candidateIndex].votes++;
    }
```

```
    function getCandidateCount() public view returns (uint256) {
        return candidates.length;
    }

    function getCandidate(uint256 index)
        public
        view
        returns (string memory, uint256)
    {
        require(index < candidates.length, "Invalid candidate");

        return (
            candidates[index].name,
            candidates[index].votes
        );
    }
}
```

5.2 Procedure and Output

Step 1: Open Remix IDE and create a new file named `Voting.sol`. Paste the smart contract code.

[Insert screenshot: Remix IDE with Voting.sol]

Step 2: Open the Solidity Compiler tab and compile the contract. A green tick indicates successful compilation.

[Insert screenshot: successful compilation]

Step 3: Open the Deploy & Run Transactions tab, select **Remix VM** as the environment, enter `["Alice", "Bob"]` in the constructor field, and click **Deploy**.

[Insert screenshot: deployment with constructor input]

Step 4: Under Deployed Contracts, call `vote` with `0` to vote for Alice.

[Insert screenshot: vote transaction for Alice]

Step 5: Switch to another test account in the Account dropdown and call `vote` with `1` to vote for Bob.

[Insert screenshot: vote transaction for Bob from second account]

Step 6: Call `getCandidate` with `0` and `1` to view each candidate's vote count.

[Insert screenshot: getCandidate output]

Step 7: Call `vote` again from an account that has already voted. The transaction is reverted.

[Insert screenshot: reverted transaction with "Already voted"]

Step 8 (Web portal): Open the web portal in Google Chrome, connect MetaMask, and log in to cast a vote. The output screens of the portal are shown in Section 5.3.

5.3 Web Portal Output

The following screens show the web portal of the reference implementation [6], connected to a local blockchain through MetaMask.

Aadhaar authentication: The voter enters their Aadhaar number and clicks Login. The "Blockchain connected" message confirms the connection to the blockchain.

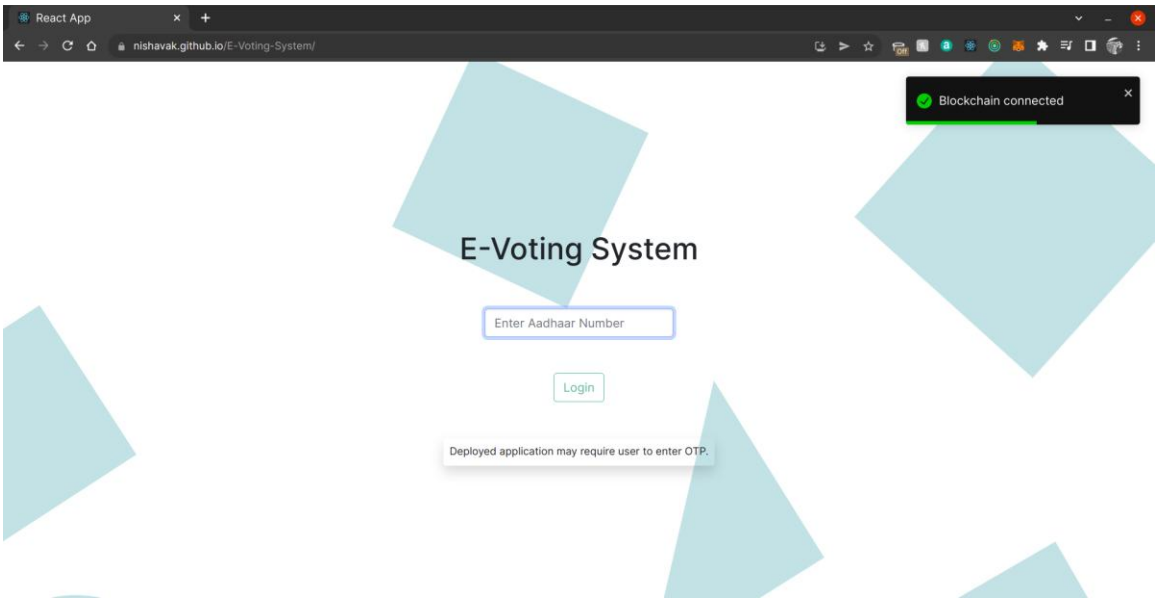


Figure 5.1: Aadhaar authentication

After successful Aadhaar authentication: The voting palette opens with the candidate list, and each candidate has a Vote link.

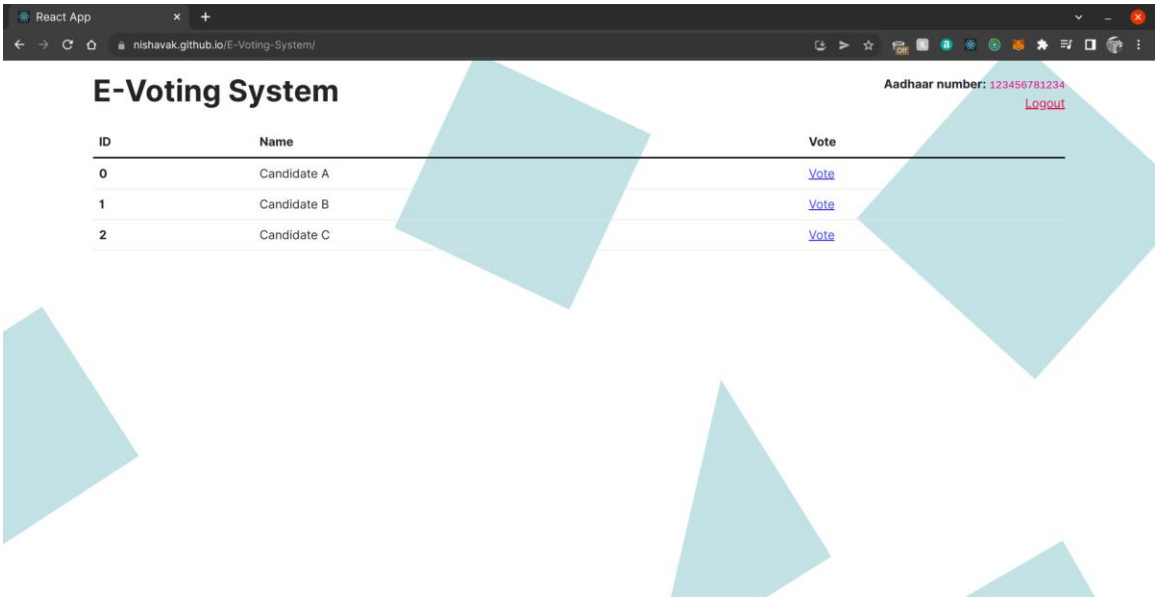


Figure 5.2: Candidate list after successful authentication

MetaMask confirmation: MetaMask asks the voter to confirm the transaction for the selected candidate, showing the estimated gas fee.

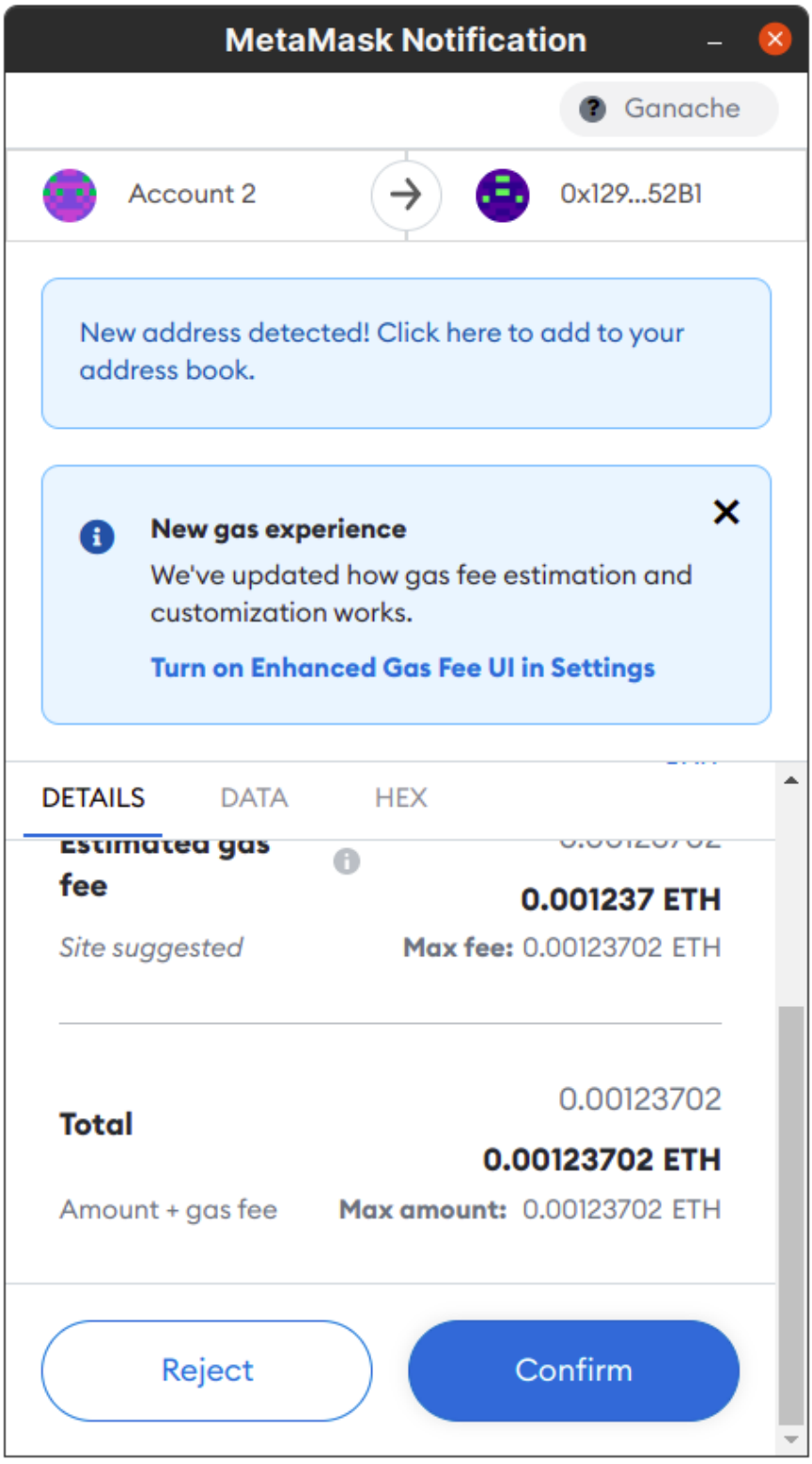


Figure 5.3: MetaMask confirmation to vote for the selected candidate

Vote confirmation: After the transaction is confirmed, the portal shows a prompt that the vote was cast for Candidate A.

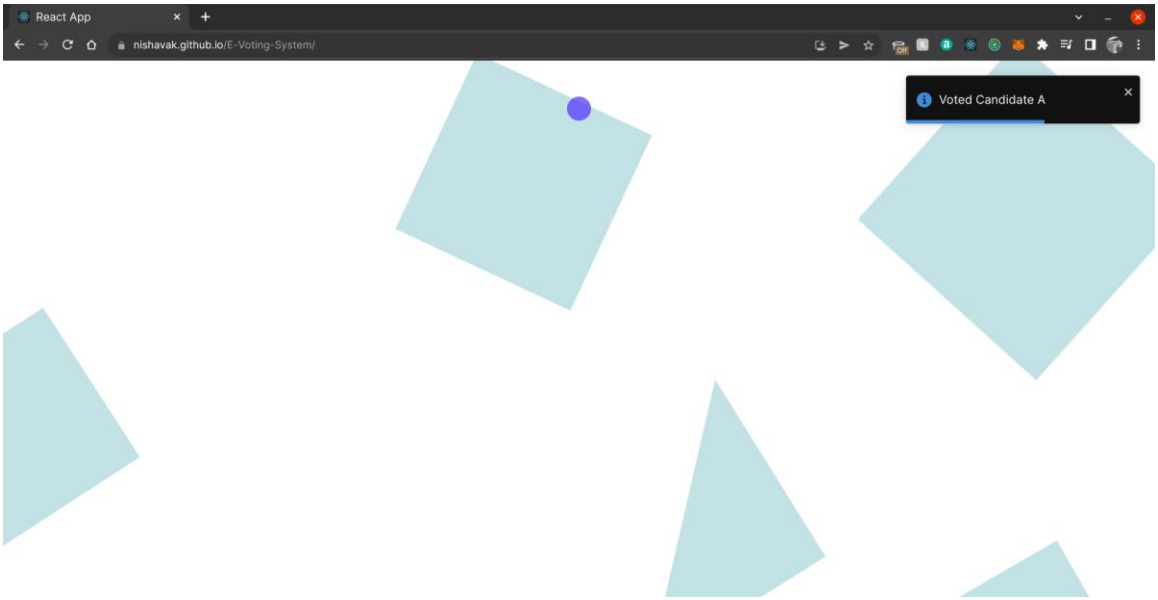


Figure 5.4: Successfully voted prompt

Vote count after voting: On logging in again, the vote count next to the candidate has increased. The portal does not log who voted for whom.

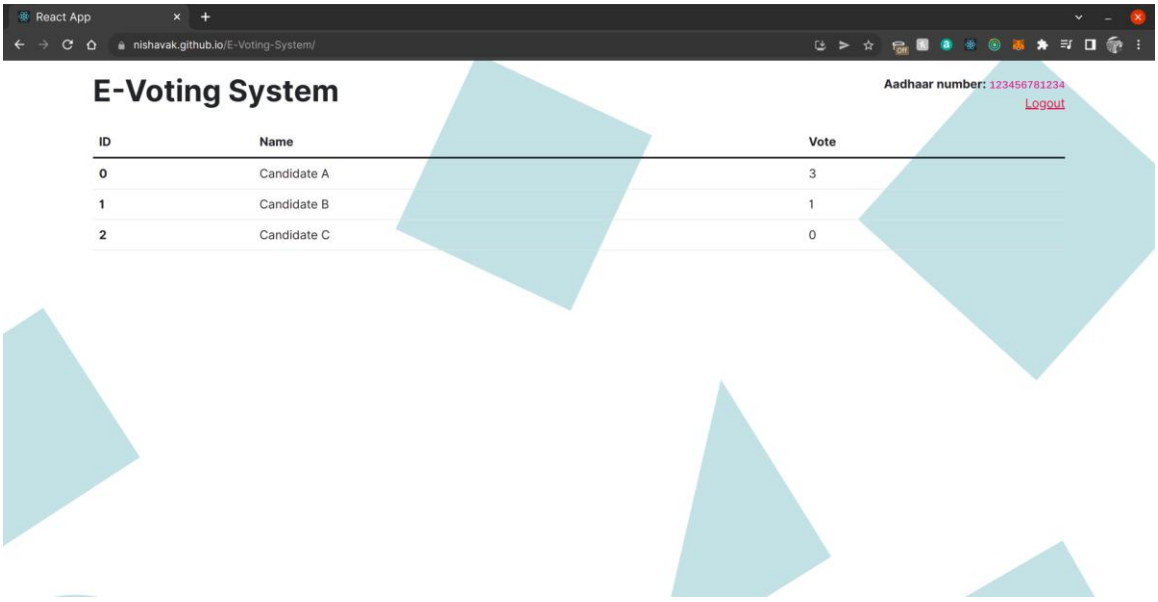


Figure 5.5: Updated vote counts for each candidate

5.4 Sample Output

```
Voting Contract Deployed Successfully

Candidate 0: Alice
Candidate 1: Bob

Alice receives one vote.
Bob receives one vote.

Alice Votes: 1
Bob Votes: 1

Second vote from the same address:
Transaction Reverted: Already voted
```

This is illustrative output; actual results depend on the transactions executed.

CHAPTER 6: CONCLUSION

The Decentralized dApp E-Voting System project successfully demonstrates the development of a basic blockchain-based voting application using Solidity and Ethereum-compatible technology. A smart contract was implemented to record votes and to prevent an address from voting more than once, and its behavior was verified through deployment and testing in Remix IDE.

By storing votes on a blockchain, the system provides transparency and tamper resistance without relying on a central authority. The use of Remix IDE, MetaMask, and a simple web frontend shows how a complete dApp can be assembled from standard tools.

Key Outcomes

- Successful implementation of a voting smart contract in Solidity
- Deployment and testing of the contract using Remix VM
- Prevention of duplicate voting through address-based validation
- Transparent, publicly readable vote counts

Learning Outcomes

- Understanding of blockchain and smart contract concepts
- Hands-on experience with Solidity and Remix IDE
- Knowledge of wallet-based interaction using MetaMask
- Experience in designing and testing a decentralized application

Future Scope

- Voter registration and eligibility checks by the owner
- Election start and end times
- Privacy-preserving voting techniques
- Deployment to a public Ethereum test network with a complete web interface

CHAPTER 7: REFERENCES

[1] Solidity Documentation, https://docs.soliditylang.org

[2] Ethereum Developer Documentation, https://ethereum.org/en/developers/docs/

[3] Remix IDE Documentation, https://remix-ide.readthedocs.io

[4] MetaMask Documentation, https://docs.metamask.io

[5] Blockchain and Smart Contract Study Material (College Notes)

[6] Ethereum-Blockchain-based-Electronic-Voting-System (reference implementation, source of the web portal screenshots), GitHub, https://github.com/riya-joshi-401/Ethereum-Blockchain-based-Electronic-Voting-System